

passgen

deterministic password generator

full documentation for project defense

client-side only (index.html)

1. idea and problem

Modern users have 20-30 online accounts: email, social media, banking, work, school. Each needs a password. Using the same password everywhere is dangerous. Remembering 30 different complex passwords is impossible. Password managers store everything in one place creating a single point of failure.

passgen offers a different approach: deterministic generation. Instead of storing passwords, it computes each one on demand using HMAC-SHA256. The user remembers one secret phrase (seed). The password for any service is computed as:

password = HMAC-SHA256(seed, service + length + counter)

Benefits:

- No storage needed: passwords are always recoverable from seed + service name
- Nothing to steal: no password database exists
- Cannot be lost: the seed is the only thing to remember
- Different passwords for every service
- Works offline: everything happens locally in the browser
- Free and open source

2. structure

Everything is in one file: index.html. It contains HTML markup, CSS styles, and JavaScript logic. No Python, no server, no databases required. The file opens in any modern browser.

2.1. html structure

The interface is organized into cards:

1. Top bar: "passgen" title, Export and Import links
2. Seed card: secret phrase input, error message area
3. Service card: service name input, list button with chip panel
4. Settings card: presets dropdown, length with arrows, 4 checkboxes, advanced panel
5. Generate button
6. Result card: login field with button, password field, strength indicator
7. Copy button
8. Import modal (hidden by default)

2.2. css design

CSS variables for colors:

- bg: #f5f3ff (light purple background)
- card: #ffffff (white card background)
- fg: #1a1a2e (dark text)
- border: #e5e7eb (light border)
- lilac: #a855f7 (purple accent)
- pink: #f472b6 (pink - weak password)
- crimson: #e11d48 (crimson - medium password)
- error: #f43f5e (red - error)

Animations: fadeUp (element entry), glow (error pulse), slideDown/slideUp (panel open/close), checkPop (checkbox toggle bounce).

Buttons: gradient background, rounded corners (11px), shadow, press animation (scale 0.97).

Cards: white, rounded corners (12px), shadow, padding.

Scrollbar hidden, scrolling works.

2.3. javascript logic

All logic is in JavaScript. Uses Web Crypto API - the browser built-in cryptography library. No external libraries needed. The code handles: password generation, settings management, export/import, event handling.

3. cryptographic algorithm

Step-by-step explanation of how passwords are generated.

3.1. hmac-sha256 generation

1. Seed is converted to bytes via TextEncoder.
2. Cryptographic key is created: `crypto.subtle.importKey("raw", keyBytes, {name:"HMAC", hash:"SHA-256"}, false, ["sign"])`
3. Base message is formed: service + length (as string, then bytes)
4. Loop until password reaches desired length:
 - a. Append counter (0, 1, 2...) to the base message
 - b. Compute HMAC: `crypto.subtle.sign("HMAC", key, message)`
 - c. For each of 32 result bytes: if byte < max, convert to character
 - d. Increment counter
5. Return the password

3.2. rejection sampling

256 (byte values) does not divide evenly by alphabet length. If the program just used `byte % len(alphabet)` for all bytes, some characters would appear more often than others (modulo bias). This weakens cryptographic strength.

Solution: $\text{max} = 256 - (256 \% \text{len}(\text{alphabet}))$. Bytes $\geq \text{max}$ are rejected. Only bytes $< \text{max}$ are used. Example: 50-char alphabet: $\text{max} = 250$. Bytes 250-255 are discarded. Each character appears exactly 5 times (250/50).

This is called rejection sampling - a standard technique in cryptography.

3.3. inversion options

First capital: the first character of the password is converted to uppercase via `.toUpperCase()`. If already uppercase, unchanged.

Last digit: a separate HMAC(`seed+"_lastdig"`, `service+length`) is computed. $\text{digit} = \text{first_result_byte} \% 10$. The last password character is replaced with this digit. Separate HMAC prevents interference with the main password.

3.4. character set alphabets

Uppercase: ABCDEFGHJKLMNPQRSTUVWXYZ (no I, O - confused with 1, 0)

Lowercase: abcdefghikmnpqrstuvwxyz (no j, l, o - confused with digits)

Digits: 0123456789

Symbols: !@#\$%&*+=-.

Defaults: uppercase + lowercase + symbols, length 10

3.5. login generation

Login uses SHA-256 (not HMAC). Input: seed + "login" + service + counter. Counter increments on each call, so each click gives a different login. First 6 hash bytes become characters.

Login alphabet: "abcdefghijklmnopqrstuvwxyz23456789"

(no i,l,o - confused with 1,0; no 0,1 - confused with letters).

Login length: exactly 6 characters.

4. interface overview

4.1. seed field

The most important field. Enter your secret phrase here. Without it, passwords cannot be recovered. When empty during generation: pink glowing border (glow animation) and error message. Press Enter to generate.

4.2. service field

Enter the service name (google, github, vk). A "list" button on the right opens a panel with service chips. Popular services have colored logo circles. User-added services have gray circles. Left click: select service. Right click on saved service: delete it.

4.3. service chips

Popular services: Google (blue G), GitHub (black GH), VK (blue V), Yandex (red Ya), Mail (blue @), iCloud (dark blue i), Dropbox (blue D), Telegram (light blue T).

User services: gray circle with "?".

Click: fills service field and applies settings.

4.4. settings

Presets: Custom, Minimal (8, letters+digits), Standard (10, no symbols), Maximum (16, everything).

Length: number input (1-99) with up/down arrow buttons.

Character sets: 4 toggle circles (uppercase, lowercase, digits, symbols).

Advanced: "First capital" and "Last digit" toggles (hidden by default).

4.5. result area

Login field (read-only) with "Generate login" button.

Password field (read-only) - monospace font, bold, centered.

Strength indicator: 3 colored bars + text recommendation.

4.6. buttons

"Generate" - purple gradient, white text, shadow. Runs HMAC generation.

"Copy" - violet gradient. Copies password to clipboard.

Both have press animation (scale 0.97) and entry animation (fadeUp).

4.7. import modal

Appears when importing JSON. Overlay with centered box. Shows service list with checkboxes (all enabled). "Cancel" and "Import" buttons.

5. all javascript functions

Detailed explanations of every function in the program.

5.1. init() -- program startup

Runs automatically when the page loads. Calls loadState() to clear old data. Removes seed field errors. Attaches event handlers: seed input clears error, Enter triggers generation, service blur remembers the current service. Restores settings for current service or sets defaults (uppercase+lowercase+symbols, length 10). Renders service chips.

5.2. loadState() -- data cleanup for security

Clears everything: localStorage, perSvc (settings), svcs (service list), resets to defaults, clears service input field, sets length to 10. Each session starts fresh. No traces left from previous use.

5.3. saveCfg() -- save settings for service

When user clicks Generate, saveCfg() remembers current checkbox states, inversion options, and length. Stores them in perSvc[serviceName]. Data is in memory only - disappears on page close.

5.4. applyCfg(cfg) -- restore service settings

When user selects a service from the list, applies its saved settings: sets length, toggles checkboxes and inversion, re-renders the interface.

5.5. toggleInv(id) -- toggle inversion options

Toggles "First capital" (id="cap") or "Last digit" (id="dig") on/off. Updates the circle display. Does NOT save - saving happens on Generate.

5.6. applyPreset() -- apply preset

Sets length and character sets based on preset choice: Minimal (8, caps+lower+digits), Standard (10, caps+lower+digits), Maximum (16, all four). "Custom" does nothing.

5.7. renderChk() -- render checkboxes

Creates 4 rows with circles and labels for each character set. Active sets get purple circles. Click toggles the value and re-renders.

5.8. renderInv() -- render inversion circles

Adds or removes CSS class "on" from inversion circles based on current values.

5.9. adj(delta) -- adjust length

Changes length by +1 or -1 on arrow click. Clamps between 1 and 99.

5.10. toggleSvcPanel() -- toggle service panel

Toggles CSS class "open" on service panel. Shown: max-height 300px. Hidden: max-height 0. Arrow rotates on toggle.

5.11. renderSvcChips() -- build service list

First shows popular services not yet in saved list, then all saved services. Calls chip() for each service.

5.12. chip() -- create service button

Creates a chip with logo. Popular services: colored circle with letter. User services: gray circle with "?". Left click: fill service field. Right click on deletable: remove from list.

5.13. gen() -- password generation (main function)

The most important function. Triggered by Generate button or Enter key.

Steps:

1. Check seed is not empty (otherwise pink error, stop)
2. Check at least one character set is selected (otherwise error, stop)
3. Build alphabet from selected sets
4. Call hmacGen() to compute HMAC-SHA256
5. Apply first capital if enabled
6. Apply last digit if enabled
7. Display password
8. Update strength indicator
9. Copy to clipboard
10. Add service to list if new
11. Save settings

5.14. hmacGen() -- cryptographic core

Computes HMAC-SHA256. Async function (browser stays responsive).

Algorithm:

1. Seed -> bytes -> HMAC key via `crypto.subtle.importKey`
2. Base message: service + length as bytes
3. $\text{max} = 256 - (256 \% \text{alphabet_length})$ for rejection sampling
4. Loop until password is long enough:
 - Add counter to message
 - Compute HMAC-SHA256 (32 bytes)
 - For each byte < max: add character to password
 - Increment counter
5. Return password

5.15. genLogin() -- login generation

Creates a 6-character login. Checks seed and service. Increments loginCounter. Computes SHA-256 of `seed+"login"+service+counter`. First 6 bytes become characters from alphabet "abcdefghijklmnopqrstuvwxyz23456789". Each click gives a different login (due to the counter).

5.16. updateStrength() -- strength evaluation

Evaluates password quality:

- Length < 10 or < 3 sets: weak (pink, 1 bar, "add symbols or increase length")
- Length >= 10 and >= 3 sets: medium (crimson, 2 bars, "good, could be longer")
- Length >= 14 and = 4 sets: strong (purple, 3 bars, "excellent password")

Shows colored bars and text recommendation.

5.17. cpy() -- copy to clipboard

Copies password to clipboard via `navigator.clipboard.writeText()`. Falls back to `execCommand("copy")` on error. Shows "Copied!" for 1.5s.

5.18. exportData() -- json export

Collects all services with settings, formats as JSON (2-space indentation), creates Blob, programmatically clicks download link. File: passgen_export.json.

5.19. importData(event) -- json import

Reads selected JSON file via FileReader. Parses and validates. If data is valid, opens the import modal with service selection.

5.20. showImportModal(d) -- import service selection

Creates a list of all services from the file with checkboxes (all enabled). If the file contains no services, shows an error message.

5.21. confirmImport() -- confirm import

Takes checked services, loads settings into perSvc, adds to svcs, sorts, re-renders chips. Shows "Import complete!" for 3 seconds.

5.22. closeImportModal() -- close import modal

Removes the modal overlay and clears temporary data.

5.23. toggleAdvanced() -- advanced panel

Expands/collapses the inversion options panel via CSS (max-height 0 -> 120px).

5.24. getCfg(svc) -- get service config

If no settings exist for a service, creates defaults. Returns settings object.

5.25. onSvcChange() -- service change handler

Updates current service and applies settings if they exist.

5.26. saveState() -- stub for security

Empty function. Previously saved to localStorage. Now does nothing (session-only mode). Kept for future compatibility.

6. error handling

1. Empty seed: pink glowing border, "Enter seed" message, generation stops.
2. No character sets: "Select at least one character type".
3. Length out of range (1-99): `adj()` clamps to boundary.
4. Invalid import file: error message shown.
5. Empty import file: "No services to import".
6. Clipboard error: fallback to `execCommand("copy")`.

7. json export/import format

```
{
  "svcs": ["google", "github"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs: array of service names.

perSvc: object with settings per service:

state: flags for capitals, lowercase, digits, symbols (1 = enabled)

inv: flags for cap (first capital), dig (last digit)

len: password length (1-99)

8. security

1. All data stays local: browser runs on localhost, nothing goes to the network.
2. Session-only mode: all settings in RAM, cleared on page reload.
3. Standard cryptography: HMAC-SHA256 (RFC 2104, FIPS 180-4). Used in banking, TLS, blockchain.
4. Rejection sampling: uniform character distribution, no modulo bias.
5. One-way: knowing the password does not reveal the seed (hash irreversibility).
6. Different passwords per service: service name is part of HMAC input.

limitations

- Does not store passwords (computes them on demand)
- Does not use salt (service name serves as salt)
- Not keylogger-proof (like any password generator)
- The seed must be kept secret and safe
- Web Crypto API requires http:// or https:// (not file://)

9. running the program

The program is a single file: index.html. Open it in any browser.

Method 1: double-click index.html -> opens in browser.

Method 2: browser -> Ctrl+O -> select index.html.

Method 3: start a local HTTP server:

```
python3 -m http.server  
then open http://localhost:8000
```

Requirements:

- Modern browser (Chrome, Firefox, Edge, Safari)
- Web Crypto API (built into all modern browsers)
- No internet required
- No Python required (optional for http server)

Important: The file should be served via http:// or https:// protocol, not file://. Web Crypto API requires a secure context. When opening via file://, cryptography may be unavailable.